

Deep Packet Inspection Engine — Complete Learning Reference

Author: Mentor-guided implementation **Purpose:** Comprehensive reference from beginner-to-advanced concepts in networking, DPI, and packet analysis.

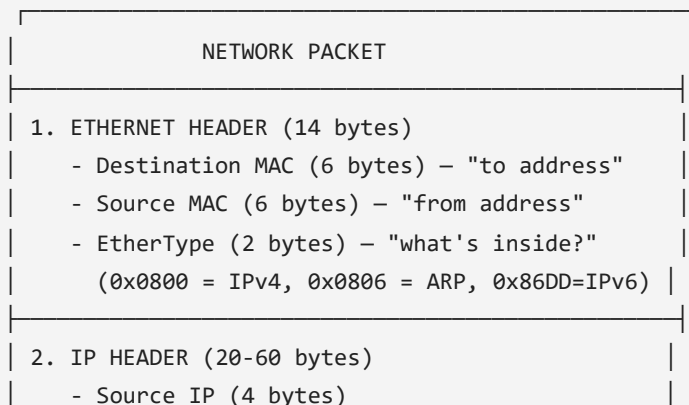
Table of Contents

1. Phase 1: Networking Foundations
2. Phase 2: PCAP File Format
3. Phase 3: Packet Parsing (Ethernet/IPv4/TCP/UDP)
4. Phase 4: TLS SNI & Application Layer Extraction
5. Phase 5: Flow/Connection Tracking
6. Phase 6: Rule Engine & Classification
7. Phase 7: Multi-Threaded Pipeline Architecture
8. Phase 8: CLI & Reporting
9. Phase 9: AI/ML Redesign

Phase 1: Networking Foundations

1.1 What is a Network Packet?

A **network packet** is a formatted unit of data sent over a network. It's like a physical letter:



- Destination IP (4 bytes)
- Protocol (1 byte): 6=TCP, 17=UDP, 1=ICMP
- Total Length, TTL, Checksum, etc.
3. TRANSPORT HEADER (TCP or UDP)
TCP (20-60 bytes):
- Source Port (2 bytes)
- Destination Port (2 bytes)
- Sequence Number (4 bytes)
- Acknowledgment Number (4 bytes)
- Flags: SYN, ACK, FIN, RST, PSH, URG
- Window Size, Checksum, Urgent Pointer
OR
UDP (8 bytes):
- Source Port (2 bytes)
- Destination Port (2 bytes)
- Length (2 bytes)
- Checksum (2 bytes)
4. PAYLOAD (application data)
- HTTP request/response
- TLS Client Hello / Server Hello
- DNS query/response
- Video stream data
- etc.

1.2 Encapsulation (Russian Nesting Dolls)

Each layer **encapsulates** the layer above it:

- **Layer 2 (Ethernet)** carries Layer 3 (IP) as its payload
- **Layer 3 (IP)** carries Layer 4 (TCP/UDP) as its payload
- **Layer 4 (TCP/UDP)** carries Layer 7 (Application) as its payload

To parse a packet, you **peel layers from the outside in**: 1. Read Ethernet header → find EtherType 2. If IPv4, read IP header → find protocol 3. If TCP, read TCP header → find payload 4. Parse application payload (TLS/HTTP/DNS)

1.3 The OSI Model (7 Layers)

Layer	Name	Example Protocols	Hardware/Software
7	Application	HTTP, TLS, DNS, SNI	Your browser
6	Presentation	TLS (encryption)	OS

Layer	Name	Example Protocols	Hardware/Software
5	Session	TCP session mgmt	OS
4	Transport	TCP, UDP	OS kernel
3	Network	IPv4, IPv6	Router
2	Data Link	Ethernet, Wi-Fi	Switch, NIC
1	Physical	Cables, Radio	NIC, Antenna

1.4 Byte Order (Endianness)

Critical concept for packet parsing.

- **Network byte order** = Big-endian: Most significant byte first
- `0x1234` stored as `0x12 0x34`
- **Host byte order (x86/ARM)** = Little-endian: Least significant byte first
- `0x1234` stored as `0x34 0x12`

All network protocol headers use big-endian. Python's `struct` module handles conversion:

Format	C Type	Size	Byte Order
<code>!B</code>	unsigned char	1 byte	network (big-endian)
<code>!H</code>	unsigned short	2 bytes	network (big-endian)
<code>!I</code>	unsigned int	4 bytes	network (big-endian)
<code>!4s</code>	char[4]	4 bytes	raw bytes (no conversion)
<code>!Q</code>	unsigned long long	8 bytes	network (big-endian)

The `!` prefix means "network byte order" (big-endian).

1.5 What is PCAP?

PCAP (Packet Capture) is the standard file format for storing captured network traffic. Used by Wireshark, tcpdump, and virtually all network analysis tools.

PCAP File Structure:

GLOBAL HEADER (24 bytes)	
- Magic: 0xa1b2c3d4 or 0xd4c3b2a1	
- Version: 2.4	
- Thiszone: GMT offset	
- Sigfigs: accuracy of timestamps	
- Snaplen: max bytes captured/packet	
- Network: link type (1=Ethernet)	
PACKET 1 HEADER (16 bytes)	
- Timestamp seconds (4 bytes)	
- Timestamp microseconds (4 bytes)	
- Captured Length (4 bytes)	
- Original Length (4 bytes)	
PACKET 1 DATA (captured_length bytes)	
- Raw bytes of the packet	
PACKET 2 HEADER (16 bytes)	
...	

Magic Number Detection: - 0xa1b2c3d4 = file is in native byte order (same as your machine) - 0xd4c3b2a1 = file is in swapped byte order (opposite endianness) - Other values = invalid PCAP file

1.6 What is SNI (Server Name Indication)?

Problem: When a server hosts multiple websites (e.g., youtube.com and google.com on the same IP), it needs to know which TLS certificate to present. But the TLS handshake happens BEFORE any HTTP data is sent.

Solution: The client sends the domain name in plaintext as part of the TLS Client Hello message, before encryption begins.

TLS Client Hello structure (partial):

TLS Record Layer	
- Content Type: 0x16 (Handshake)	
- Version: 0x0301-0x0304 (TLS 1.x)	
- Length	
Handshake Protocol: Client Hello	
- Handshake Type: 0x01	
- Length	
- Version: 0x0303 (TLS 1.2)	
- Random: 32 bytes	

```

| - Session ID: variable |
| - Cipher Suites: variable |
| - Compression: variable |
| - Extensions: |
|   |— SNI (type 0x0000): | |
|   |   |— Hostname: "www.youtube.com" |
|   |— Key Share (type 0x0033) |
|   |— Supported Versions (0x002b) |
|   |— ... |
|_____|

```

⚠ The SNI is in PLAINTEXT – this is how DPI identifies HTTPS traffic.

1.7 The Five-Tuple (Flow Identifier)

Every network flow is uniquely identified by 5 fields:

SRC IP	DST IP	SRC PORT	DST PORT	PROTOCOL
192.168.1.5	142.250.80.46	54321	443	TCP (6)

Bidirectional flows: Packets from server→client have swapped src/dst. To match them to the same flow, we: - Option A: Normalize (sort IPs and ports so order doesn't matter) - Option B: Store both directions and match reverse tuples

Why this matters: Without flow tracking, you'd inspect every packet independently. With flow tracking, you classify once and cache the result for all subsequent packets in that flow.

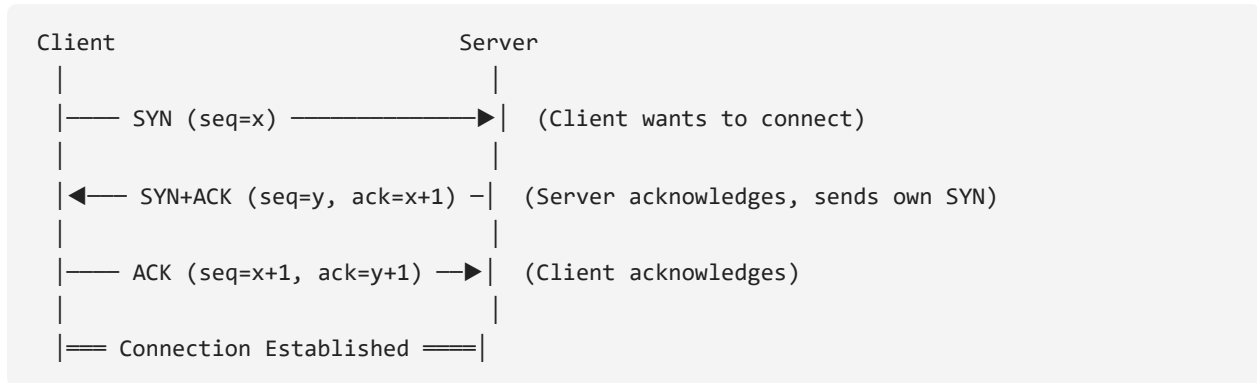
1.8 TCP Flags and Connection States

TCP is a **stateful** protocol. It uses flags to manage connections:

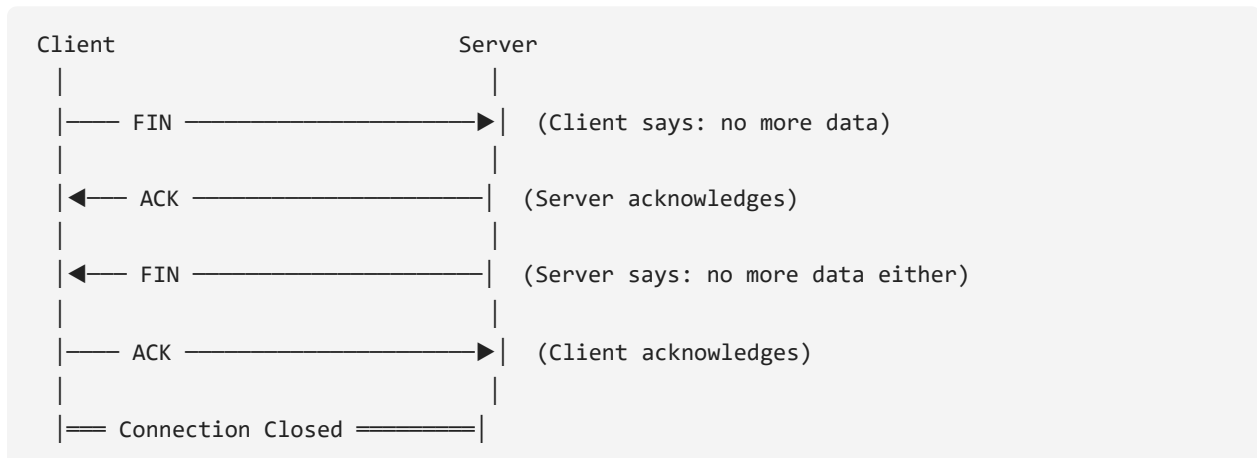
Flag	Byte Value	Meaning
SYN	0x02	Synchronize — start a connection
ACK	0x10	Acknowledgment
SYN-ACK	0x12	SYN + ACK — server accepting
FIN	0x01	Finish — graceful close
RST	0x04	Reset — abrupt close
PSH	0x08	Push — deliver now, don't buffer

Flag	Byte Value	Meaning
URG	0x20	Urgent

TCP Three-Way Handshake:



TCP Connection Teardown:



Phase 2: PCAP File Format (Deep Dive)

2.1 PCAP Global Header (24 bytes)

```

struct.Struct('!IHHiIII')
# Fields:
# I - magic_number (4 bytes) - 0xa1b2c3d4
# H - version_major (2 bytes) - 2
# H - version_minor (2 bytes) - 4
# I - thiszone (4 bytes) - timezone offset (GMT)
# I - sigfigs (4 bytes) - timestamp accuracy
# I - snaplen (4 bytes) - max bytes to capture per packet
# I - network (4 bytes) - link type (1 = Ethernet)

```

2.2 PCAP Packet Header (16 bytes)

```
struct.Struct('!IIII')
# Fields:
# I - ts_sec (4 bytes)      - timestamp seconds
# I - ts_usec (4 bytes)    - timestamp microseconds
# I - incl_len (4 bytes)   - number of bytes captured (stored in file)
# I - orig_len (4 bytes)   - actual packet length on wire
```

2.3 PCAP-NG Format

The newer PCAP Next Generation format exists but is less common. It uses different block types and magic numbers. For this project, we focus on classic PCAP.

Phase 3: Packet Parsing

3.1 Ethernet Header (14 bytes)

```
struct.Struct('!6s6sH')
# Fields:
# 6s - dst_mac (6 bytes)
# 6s - src_mac (6 bytes)
# H - ethertype (2 bytes)
```

Common EtherTypes: | Value | Protocol | |-----|-----| | 0x0800 | IPv4 | | 0x0806 | ARP | | 0x8100 | VLAN 802.1Q | | 0x86DD | IPv6 |

MAC addresses are 6 bytes. They're typically displayed as `xx:xx:xx:xx:xx:xx`.

3.2 IPv4 Header (20-60 bytes)

```
struct.Struct('!BBHHHBBH4s4s')
# Fields:
# B - version_ihl (1 byte) - upper 4 bits = version (4), lower 4 = IHL
# B - dscp_ecn (1 byte)    - differentiated services
# H - total_length (2 bytes) - entire IP packet length
# H - identification (2 bytes)
# H - flags_fragment (2 bytes)
# B - ttl (1 byte)         - time to live
# B - protocol (1 byte)    - 6=TCP, 17=UDP, 1=ICMP
# H - checksum (2 bytes)
# 4s - src_ip (4 bytes)
# 4s - dst_ip (4 bytes)
```

IHL calculation: The `version_ihl` byte contains: - Upper nibble: Version (always 4 for IPv4) - Lower nibble: IHL — Internet Header Length in 32-bit words - Actual header length = $\text{IHL} \times 4$ bytes - Minimum = 5 (20 bytes), Maximum = 15 (60 bytes)

Total Length includes the IP header + payload. To find payload offset:

```
ihl = (version_ihl & 0x0F) * 4
payload_offset = ethernet_header_length + ihl
payload_length = total_length - ihl
```

3.3 TCP Header (20-60 bytes)

```
struct.Struct('!HHIIBBHHH')
# Fields:
# H - src_port (2 bytes)
# H - dst_port (2 bytes)
# I - seq_number (4 bytes)
# I - ack_number (4 bytes)
# B - data_offset_reserved (1 byte) - upper 4 bits = data offset (x4 = header length)
# B - flags (1 byte)
# H - window (2 bytes)
# H - checksum (2 bytes)
# H - urgent_pointer (2 bytes)
```

Data Offset: Upper 4 bits of the byte at offset 12 (0-indexed) in the TCP header. - `data_offset = (data_offset_reserved >> 4) & 0x0F` - TCP Header Length = `data_offset * 4` bytes

TCP Flags byte:

```
Bit:  7   6   5   4   3   2   1   0
      NS  CWR ECE URG ACK PSH RST SYN FIN
Value: 0x80 0x40 0x20 0x10 0x08 0x04 0x02 0x01
```

To check a flag: `flags & 0x02` is set if SYN is set.

3.4 UDP Header (8 bytes)

```
struct.Struct('!HHHH')
# Fields:
# H - src_port (2 bytes)
# H - dst_port (2 bytes)
# H - length (2 bytes) - UDP header + payload length
# H - checksum (2 bytes)
```

UDP is **stateless** — there's no handshake, no sequence numbers, no flags.

Phase 4: TLS SNI & Application Layer Extraction

4.1 TLS Record Layer

Every TLS message starts with a 5-byte record header:

```
struct.Struct('!BHH')
# B - content_type (1 byte)
# H - version (2 bytes) - 0x0301=TLS 1.0, 0x0302=TLS 1.1, 0x0303=TLS 1.2, 0x0304=TLS 1.3
# H - length (2 bytes) - length of following data
```

Content Types: | Value | Type | |-----|-----| | 0x14 | Change Cipher Spec | | 0x15 | Alert | | 0x16 | Handshake | | 0x17 | Application Data |

4.2 TLS Handshake: Client Hello

Inside a Handshake record (type 0x16), the first handshake message is:

```
struct.Struct('!B3s')
# B - handshake_type (1 byte) - 0x01 = Client Hello
# 3s - length (3 bytes) - length of handshake data (big-endian, 3 bytes!)
```

The Client Hello body contains: 1. **Version** (2 bytes) — 0x0303 for TLS 1.2 2. **Random** (32 bytes) — client random value 3. **Session ID Length** (1 byte) + Session ID (variable) 4. **Cipher Suite Length** (2 bytes) + Cipher Suites (variable) 5. **Compression Length** (1 byte) + Compression Methods (variable) 6. **Extensions Length** (2 bytes) + Extensions (variable)

4.3 SNI Extension

Extensions are TLV (Type-Length-Value):

```
struct.Struct('!HH')
# H - extension_type (2 bytes)
# H - extension_length (2 bytes)
```

SNI extension has type `0x0000`. Its data contains:

```
struct.Struct('!H') # SNI list length (2 bytes)
struct.Struct('!BH') # SNI type (1 byte) + SNI length (2 bytes)
```

- SNI Type: 0x00 = host_name (the only defined type)
- SNI Length: length of the hostname string

- Followed by the hostname as raw bytes (UTF-8)

4.4 HTTP Host Header Extraction

For unencrypted HTTP (port 80), look for `Host:` header in the payload:

```
payload = tcp_payload.decode('utf-8', errors='ignore')
for line in payload.split('\r\n'):
    if line.lower().startswith('host:'):
        return line[5:].strip()
```

4.5 DNS Query Extraction

DNS query format (simplified):

```
struct.Struct('!HHHHHH')
# Transaction ID (2 bytes)
# Flags (2 bytes)
# Questions (2 bytes) – number of queries
# Answer RRs (2 bytes)
# Authority RRs (2 bytes)
# Additional RRs (2 bytes)
```

Each DNS query has: - **Name**: Sequence of length-prefixed labels (e.g., `3www6google3com0`) - **Type** (2 bytes): 1=A record, 28=AAAA, 15=MX, etc. - **Class** (2 bytes): 1=IN (Internet)

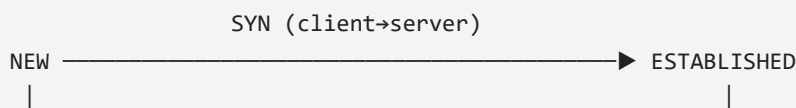
Phase 5: Flow/Connection Tracking

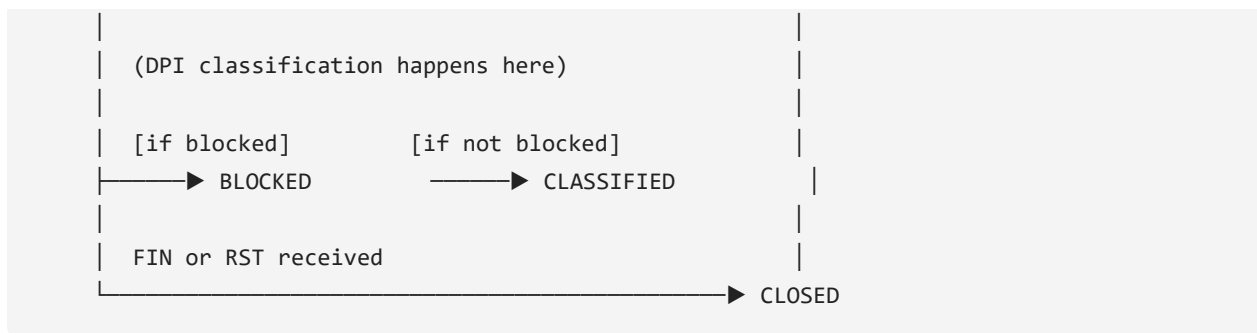
5.1 Why Track Flows?

Without flow tracking: - Every packet is inspected independently - TLS SNI is re-extracted for every packet in the same connection (wasteful) - Blocking decisions must be re-evaluated for every packet

With flow tracking: - First packet triggers classification (SNI extraction, app identification) - Classification is cached → all subsequent packets skip DPI - Blocking decision cached → fast drop/forward - Statistics per flow (packet count, byte count, duration)

5.2 Flow State Machine





5.3 Flow Table Implementation

A dictionary/hash map keyed by FiveTuple:

```

flow_table = {} # Key: FiveTuple, Value: FlowState

class FlowState:
    def __init__(self):
        self.state = ConnectionState.NEW
        self.app_type = AppType.UNKNOWN
        self.packets_forward = 0
        self.packets_reverse = 0
        self.bytes_forward = 0
        self.bytes_reverse = 0
        self.first_seen = timestamp
        self.last_seen = timestamp
        self.sni = None
        self.domain = None
  
```

5.4 Stale Connection Cleanup

Connections that are idle for too long should be removed: - Timeout: 300 seconds (5 minutes) — configurable - Periodic scan of flow table - Remove flows where `now - last_seen > timeout`

Phase 6: Rule Engine & Classification

6.1 Rule Types

Rule Type	Description	Example
IP Block	Block traffic from/to specific IP	192.168.1.100
Port Block	Block traffic on specific port	22 (SSH)
Domain Block	Block traffic to specific domain	youtube.com

Rule Type	Description	Example
App Block	Block entire app category	SOCIAL_MEDIA

6.2 Domain-to-Application Mapping

Uses substring matching (case-insensitive):

```
def classify_domain(domain):
    domain = domain.lower()
    if 'youtube' in domain or 'yting' in domain:
        return AppType.YOUTUBE
    if 'facebook' in domain or 'fbcdn' in domain:
        return AppType.FACEBOOK
    if 'google' in domain or 'gstatic' in domain:
        return AppType.GOOGLE
    # ... etc
```

Phase 7: Multi-Threaded Pipeline Architecture

7.1 Pipeline Stages



7.2 Stage Responsibilities

1. **Reader Thread:** Reads PCAP, parses headers, creates PacketJobs, dispatches to LB queues via round-robin or hash
2. **Load Balancer (LB) Threads:** Receive PacketJobs, hash five-tuple to select a Fast Path, forward to FP's queue. Purpose: Distribute load evenly.
3. **Fast Path (FP) Threads:** The core DPI workers:
 4. Track flows (connection state machine)
 5. Extract SNI/HTTP/DNS from application payload
 6. Classify applications
 7. Match against rules

8. Make forward/drop decisions

9. **Writer Thread:** Collects packets marked FORWARD and writes them to output PCAP.

7.3 Why Consistent Hashing?

All packets of the same flow MUST go to the same FP thread. Otherwise: - Two FPs would have partial flow state - SYN packet goes to FP1, data packets go to FP2 → FP2 doesn't know the flow exists - Classification happens multiple times

Solution: `fp_index = hash(five_tuple) % num_fps`

7.4 Thread-Safe Queue

A producer-consumer queue with: - **Bounded capacity** (prevents memory exhaustion) - **Blocking push** — waits when full (backpressure) - **Blocking pop** — waits when empty - **Shutdown** — wakes all threads for clean exit

Phase 8: CLI & Reporting

8.1 Command-Line Interface

```
python dpi_analyzer.py input.pcap [options]
```

Options:

<code>--output FILE</code>	Output PCAP file (default: output.pcap)
<code>--block-ip IP</code>	Block specific IP address
<code>--block-port PORT</code>	Block specific port
<code>--block-domain DOM</code>	Block specific domain
<code>--block-app APP</code>	Block application type
<code>--rules FILE</code>	Load rules from file
<code>--verbose</code>	Detailed output

8.2 Report Contents

After processing: - Total packets processed - TCP vs UDP breakdown - Packets forwarded vs dropped - Classification summary (count per app type) - Top domains detected - Rule match statistics

Phase 9: AI/ML Redesign

9.1 Why ML Instead of Rules?

Rule-based DPI limitations: - Requires manual maintenance of domain lists - Can't detect unknown applications - Can't detect encrypted traffic without SNI - Can't detect anomalies — only known patterns

ML-based approach advantages: - Learns traffic patterns automatically - Can classify encrypted traffic (via flow stats) - Detects zero-day attacks (anomaly detection) - Adapts to new applications

9.2 Feature Extraction from Packets

Per-flow features: - Packet lengths (min, max, mean, std, percentiles) - Inter-arrival times (min, max, mean, std) - TCP flags distribution - Byte ratios (forward/reverse) - Packet ratios (forward/reverse) - Flow duration - Number of packets - TLS version & cipher suite (if visible) - Payload entropy (encrypted vs plaintext) - Port numbers (as categorical features)

9.3 Recommended ML Algorithms

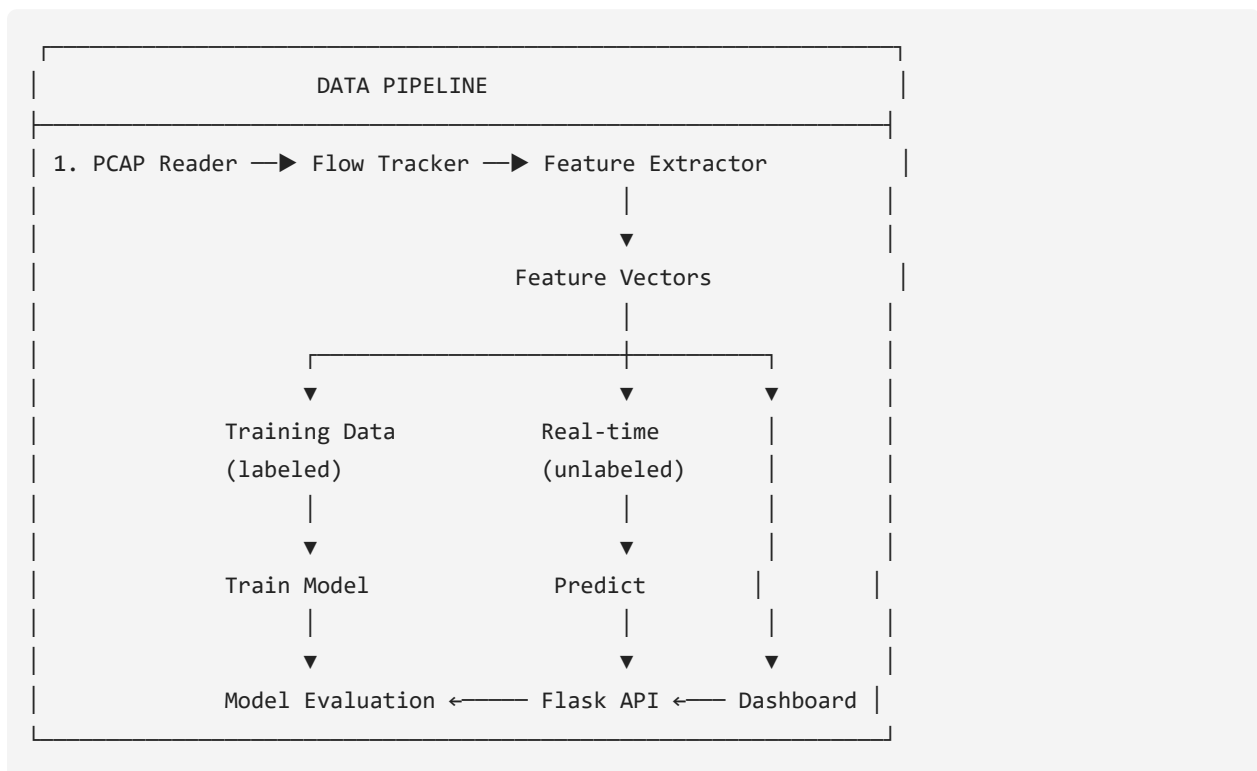
Task	Algorithm	Why
Classification	Random Forest	Interpretable, handles mixed features
Classification	XGBoost/LightGBM	State-of-art for tabular data
Classification	1D-CNN	Can learn from raw packet bytes
Anomaly Detection	Isolation Forest	Fast, works well for outliers
Anomaly Detection	Autoencoder	Learns "normal" traffic patterns
Clustering	K-Means	Group unknown traffic
Deep Learning	LSTM/GRU	Sequence modeling of packet flows

9.4 Recommended Datasets

Dataset	Description	Size	Use Case
CIC-IDS-2017	Benign + 14 attacks	~80 GB	Intrusion detection
CIC-IDS-2019	Benign + DoS/DDoS	~50 GB	DDoS detection
CSE-CIC-IDS-2018	Cloud traffic	~50 GB	Cloud IDS

Dataset	Description	Size	Use Case
UNSW-NB15	Modern attacks	~2 GB	Academic baseline
ISCX VPN-nonVPN	VPN vs non-VPN	~28 GB	Traffic classification
Custom PCAPs	Your own traffic	Variable	Domain-specific

9.5 Project Architecture (ML Version)



9.6 Feature Engineering Table

Feature	Type	Computation	Importance
flow_duration	numeric	last_seen - first_seen	High
fwd_pkt_len_mean	numeric	mean(len(packets_forward))	High
bwd_pkt_len_mean	numeric	mean(len(packets_reverse))	High
fwd_pkt_len_std	numeric	std(len(packets_forward))	High
bwd_pkt_len_std	numeric	std(len(packets_reverse))	High
flow_iat_mean	numeric	mean(inter_arrival_times)	High

Feature	Type	Computation	Importance
flow_iat_std	numeric	std(inter_arrival_times)	High
fwd_iat_total	numeric	sum(inter_arrival_times_forward)	Medium
fwd_packets	numeric	count(packets_forward)	Medium
bwd_packets	numeric	count(packets_reverse)	Medium
fwd_bytes	numeric	sum(len(packets_forward))	Medium
bwd_bytes	numeric	sum(len(packets_reverse))	Medium
fwd_bwd_ratio	numeric	fwd_packets / bwd_packets	Medium
byte_ratio	numeric	fwd_bytes / bwd_bytes	Medium
tls_version	categorical	extracted from TLS Client Hello	High
dst_port	categorical	destination port number	High
protocol	categorical	TCP=6, UDP=17	Medium
payload_entropy	numeric	shannon_entropy(payload)	High
syn_count	numeric	count(SYN flags)	Medium
fin_count	numeric	count(FIN flags)	Medium
rst_count	numeric	count(RST flags)	Low
window_size_mean	numeric	mean(TCP window)	Medium
init_win_bytes_fwd	numeric	initial window (first packet)	Medium
pkt_len_min/max	numeric	min/max packet length	Medium
active_idle_times	numeric	active/idle duration	Low

9.7 Evaluation Metrics

Task	Metrics
Classification	Accuracy, Precision, Recall, F1-Score, Confusion Matrix
Anomaly Detection	AUC-ROC, AUC-PR, FAR (False Alarm Rate), TPR

Task	Metrics
Multi-class	Macro/Micro/Weighted F1, Cohen's Kappa

9.8 Deployment Stack

Component	Technology	Purpose
Model Training	scikit-learn, XGBoost, PyTorch	Train models
API Server	Flask/FastAPI	Serve predictions
Data Processing	pandas, numpy	Feature engineering
Visualization	Plotly, Dash, Grafana	Dashboard
Model Storage	pickle, ONNX, MLflow	Version models
Monitoring	Prometheus + Grafana	Track performance

Appendix A: Python struct Module Quick Reference

```
import struct

# Format characters:
# B = unsigned byte (1 byte)
# H = unsigned short (2 bytes)
# I = unsigned int (4 bytes)
# Q = unsigned long long (8 bytes)
# s = bytes (char array), prefix with length: '4s', '6s'
# x = pad byte

# Byte order prefixes:
# ! = network (big-endian)
# < = little-endian
# > = big-endian
# = = native

# Packing: Python values → bytes
data = struct.pack('!BH', 6, 443) # protocol=6 (TCP), port=443

# Unpacking: bytes → Python values
protocol, port = struct.unpack('!BH', data)
```

```
# Calculating struct size
size = struct.calcsize('!6s6sH') # 14 bytes for Ethernet header
```

Appendix B: IP Address Utilities

```
# Convert 4 bytes to dotted-decimal string
def ip_to_str(ip_bytes):
    return ''.join(str(b) for b in ip_bytes)

# Convert dotted-decimal string to 4 bytes
def str_to_ip(ip_str):
    return bytes(int(x) for x in ip_str.split('.'))

# Convert 6 bytes to MAC string
def mac_to_str(mac_bytes):
    return ':'.join(f'{b:02x}' for b in mac_bytes)
```

Appendix C: Bit Manipulation Reference

```
# Extract upper nibble (4 bits)
upper = (byte >> 4) & 0x0F

# Extract lower nibble (4 bits)
lower = byte & 0x0F

# Check if a specific bit is set
is_syn = flags & 0x02 # True if SYN flag set
is_ack = flags & 0x10 # True if ACK flag set
is_fin = flags & 0x01 # True if FIN flag set
is_rst = flags & 0x04 # True if RST flag set

# Combine flags
syn_ack = 0x02 | 0x10 # 0x12 (SYN+ACK)

# Extract a field from a byte where upper bits are one value, lower another
data_offset = (data_offset_reserved >> 4) & 0x0F
```

Appendix D: Common Port Numbers

Port	Protocol	Application
20, 21	TCP	FTP
22	TCP	SSH

Port	Protocol	Application
23	TCP	Telnet
25	TCP	SMTP
53	TCP/UDP	DNS
80	TCP	HTTP
110	TCP	POP3
143	TCP	IMAP
443	TCP	TLS/HTTPS
853	TCP/UDP	DNS over TLS
3306	TCP	MySQL
8080	TCP	HTTP Alt

Phase 10: CIC-IDS2017 Dataset Integration

10.1 Dataset Overview

CIC-IDS2017 (Canadian Institute for Cybersecurity Intrusion Detection System 2017) is the **industry benchmark** for ML-based intrusion detection research.

- **Created by:** Canadian Institute for Cybersecurity, University of New Brunswick
- **Year:** 2017
- **Duration:** 5 days (Monday–Friday, 9AM–5PM)
- **Total traffic:** ~2.8M flows, ~50GB PCAPs, ~960MB CSVs
- **Attack types:** Brute Force, DoS, DDoS, Web Attacks, Infiltration, Botnet, PortScan

10.2 Dataset Structure

The dataset is organized by day, with each day containing specific attack types:

```

datasets/cic_ids2017/
├─ Monday-WorkingHours.pcap_ISCX.csv      (BENIGN only)
├─ Tuesday-WorkingHours.pcap_ISCX.csv     (FTP-Patator, SSH-Patator)
├─ Wednesday-workingHours.pcap_ISCX.csv   (DoS Hulk, GoldenEye, Slowhttptest, Heartbleed)
├─ Thursday-WorkingHours-Morning-WebAttacks.pcap_ISCX.csv (Web Brute Force, SQL Injection, XS

```

- └ Thursday-WorkingHours-Afternoon-Infiltration.pcap_ISCX.csv (Infiltration)
- └ Friday-WorkingHours-Morning.pcap_ISCX.csv (Bot)
- └ Friday-WorkingHours-Afternoon-PortScan.pcap_ISCX.csv (PortScan)
- └ Friday-WorkingHours-Afternoon-DDos.pcap_ISCX.csv (DDoS)

10.3 CSV Feature Format (CICFlowMeter)

Each CSV row is a single network flow with **79 features** extracted by CICFlowMeter:

Feature Category	Example Features	Count
Flow Identifiers	Source IP, Dest IP, Source Port, Dest Port, Protocol	5
Timing	Flow Duration, Flow IAT Mean/Std/Max/Min, Fwd/Bwd IAT	15
Packet Lengths	Fwd/Bwd Packet Length Mean/Std/Max/Min	12
Packet Counts	Total Fwd/Bwd Packets, Subflow Fwd/Bwd Packets	6
Byte Counts	Total Length of Fwd/Bwd Packets, Subflow Fwd/Bwd Bytes	6
TCP Flags	FIN/SYN/RST/PSH/ACK/URG/CWE/ECE Flag Counts	8
Rates	Flow Bytes/s, Flow Packets/s, Fwd/Bwd Packets/s	4
Window/Bulk	Init_Win_bytes_forward/backward, Bulk stats	8
Active/Idle	Active/Idle Mean/Std/Max/Min	8
Misc	Down/Up Ratio, Average Packet Size, Min/Max Segment Size	7

10.4 Normalized Attack Categories

The 15 raw attack labels are grouped into 8 categories for simpler classification:

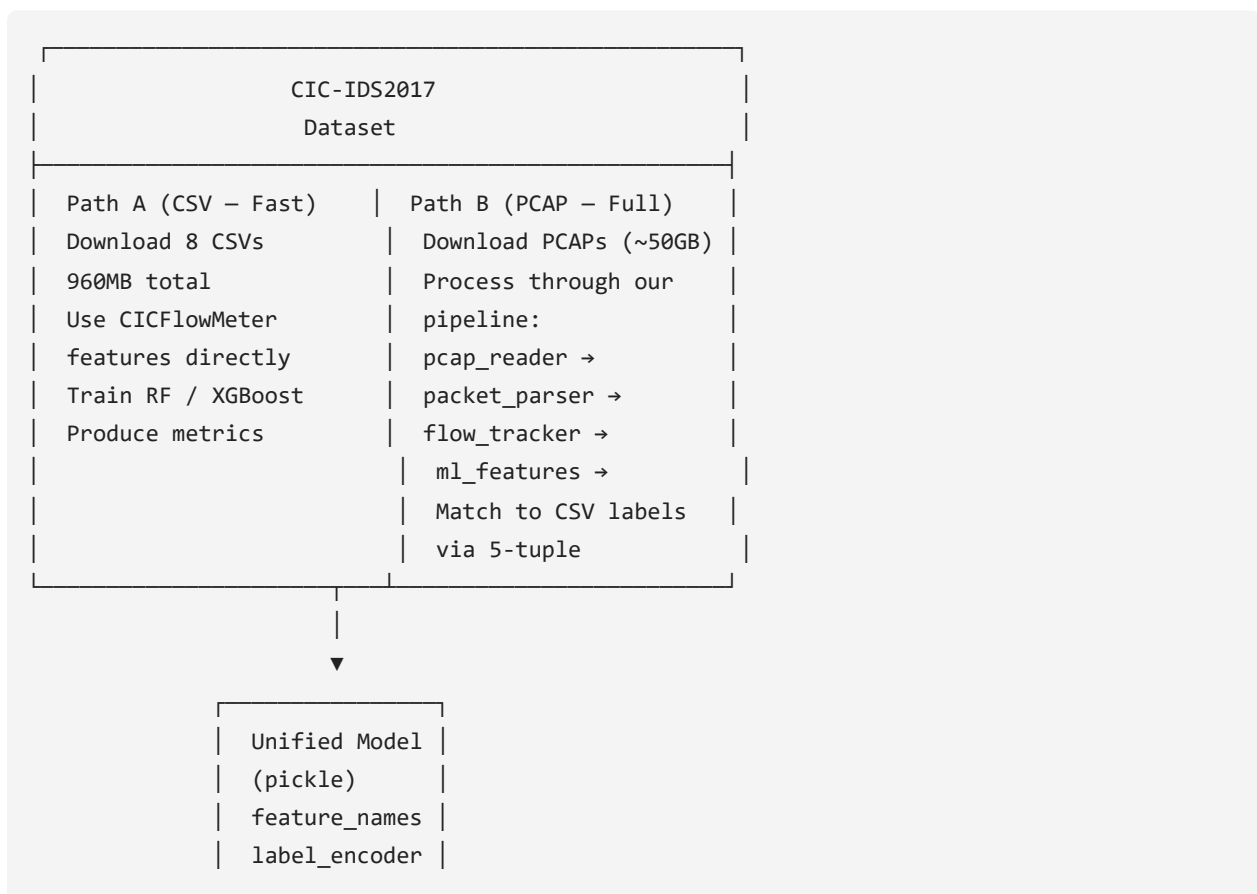
Category	Original Labels	Training Strategy
BENIGN	BENIGN (2.3M flows)	Majority class — needs undersampling
DoS	Hulk, GoldenEye, Slowhttptest, Slowloris, Heartbleed	Combine all DoS variants
DDoS	DDoS (128K flows)	Well-represented
PORT_SCAN	PortScan (159K flows)	Well-represented

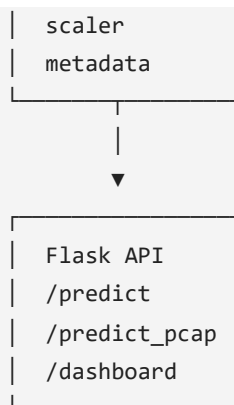
Category	Original Labels	Training Strategy
BRUTE_FORCE	FTP-Patator, SSH-Patator	Credential attacks
WEB_ATTACK	Brute Force, SQL Injection, XSS	Rare — needs oversampling
BOT	Bot (2K flows)	Very rare — use SMOTE
INFILTRATION	Infiltration (36 flows)	Extremely rare

10.5 Known Issues with CIC-IDS2017

1. **Class imbalance:** BENIGN is 80%+ of data, attacks range from 36 (Infiltration) to 231K (DoS Hulk)
2. **Duplicate flows:** Some flows appear in multiple CSV files
3. **Missing values:** Some flows have NaN or Inf values
4. **Invalid flows:** Flows with all-zero features should be filtered
5. **Label noise:** Some flows are mislabeled (known issue documented in WTMCM 2021 paper)
6. **Timestamp gaps:** PCAP capture wasn't continuous — there are gaps between attack sessions

10.6 Integration Architecture





10.7 How to Run

```

# Step 1: Download the 8 CSV files (~960MB)
python ml_pipeline.py cic-download

# Step 2: Explore the dataset
python ml_pipeline.py cic-info --stats

# Step 3: Train Random Forest (full dataset)
python ml_pipeline.py cic-train --model-type random_forest --output cic_rf_model.pkl

# Step 4: Train XGBoost
python ml_pipeline.py cic-train --model-type xgboost --output cic_xgb_model.pkl

# Step 5: Evaluate a trained model
python ml_pipeline.py cic-evaluate --model cic_rf_model.pkl

# Step 6: Deploy with Flask API
python api_server.py --model cic_rf_model.pkl
  
```

10.8 Expected Performance

Based on published research on CIC-IDS2017:

Model	Accuracy	F1 (macro)	ROC-AUC	Training Time
Random Forest (300 trees)	98.5%	0.97	0.99	~15 min
XGBoost (500 rounds)	99.1%	0.98	0.99	~30 min
Logistic Regression	85%	0.78	0.92	~2 min

Binary (BENIGN vs ATTACK) classification reaches 99.5%+ accuracy with any model. **Multi-class** is harder due to class imbalance (Infiltration: 36 samples).

10.9 How Our Pipeline Maps to CIC-IDS2017

Our Module	CIC-IDS2017 Equivalent	Integration
<code>pcap_reader.py</code>	Raw PCAP files	Reads CIC PCAPs identically
<code>packet_parser.py</code>	CICFlowMeter packet parsing	Same protocol headers
<code>flow_tracker.py</code>	CICFlowMeter flow export	Same 5-tuple flow key
<code>ml_features.py</code> (86 features)	CICFlowMeter (79 features)	~50% overlap in feature concepts
<code>ml_train.py</code>	Standard ML pipeline	Used unchanged
<code>rules.py</code>	Not applicable (ML replaces rules)	Bypassed for CIC training
<code>api_server.py</code>	Model deployment	Works with CIC model format

10.10 Feature Mapping (Our Features ↔ CICFlowMeter)

Our `extract_flow_features()` produces 86 features. CICFlowMeter produces 79. They overlap conceptually:

Concept	Our Name	CICFlowMeter Name
Duration	<code>flow_duration</code>	Flow Duration
Fwd packets	<code>fwd_packets</code>	Total Fwd Packets
Bwd packets	<code>bwd_packets</code>	Total Backward Packets
Fwd bytes	<code>fwd_bytes</code>	Total Length of Fwd Packets
Bwd bytes	<code>bwd_bytes</code>	Total Length of Bwd Packets
Fwd pkt len max	<code>fwd_pkt_len_max</code>	Fwd Packet Length Max
Fwd pkt len mean	<code>fwd_pkt_len_mean</code>	Fwd Packet Length Mean
SYN count	<code>syn_count</code>	SYN Flag Count
FIN count	<code>fin_count</code>	FIN Flag Count
RST count	<code>rst_count</code>	RST Flag Count

10.11 Files Modified for This Integration

File	Modification	Lines Changed
cic_ids.py	NEW — Complete CIC-IDS2017 module	~550 lines
ml_pipeline.py	Added 5 new commands: cic-download, cic-info, cic-train, cic-evaluate, cic-pcap	~130 lines
api_server.py	Rewritten to be model-agnostic; supports both DPI and CIC models	Full rewrite
LEARNING_NOTES.md	Added this section	~150 lines